

Blockchain Lab Practical Guided Exercises to Support the Coursework Assignment

Introduction

The goal of these practical sessions is to create an offline Blockchain application. This application can be developed in a language of your choice, although it is **highly recommended** to use C#. This brief operates on the assumption that C# is used and resources such as the template code are in C#. C# is an object-oriented language and is very similar to Java, and it should be relatively easy to pick up. Visual Studio is the recommended IDE for developing in C#, the latest version can be [found here](#).

Along with this brief you should find a visual studio project which will provide the template code for the assignment. This code should be built upon during the practicals. The template makes use of Windows Forms for the UI. Windows Forms is simple and quick to use, meaning you can focus on the technical aspects more than the appearance. This template also includes ready-made classes that will be used to develop features.

Practical Guided Exercises and Assignment

It is expected that you will complete the first 5 parts of this brief during the practical sessions. The assignment tasks are based on the application you will have made in the first 5 parts. Marks for the assignment are allocated as follows:

Table 1: Marking Scheme

	Evidence of implementation	Report	Total
Part 1 – Project Setup	3	2	5
Part 2 – Blocks and the Blockchain	5	5	10
Part 3 – Transactions and Digital Signatures	5	5	10
Part 4 – Consensus Algorithms (Proof-of-Work)	10	10	20
Part 5 – Validation	10	10	20
Part 6 – Assignment Tasks	15	20	35
	48	52	(100)

For each part in this brief, you are expected to provide evidence of implementation alongside a report which details your understanding of the topic.

In part 6 of the assignment, you are only expected to complete 3 out of 4 of the sections to achieve the maximum of 35 marks. However, if you choose to do all four you may be able to score more marks (**not** more than 35).

This brief will require students to develop a basic offline Blockchain application that includes the following:

- Blockchain made of cryptographically connected Blocks;
- Transaction generation – including digital signature via asymmetric encryption;
- A Proof-of-Work consensus algorithm – including hashing and threading;
- Validation methods to ensure the Blockchain is valid;
- A basic UI that can verify the implementation of the above features.

The following instructions are built to ease you into programming with C# and Windows Forms and gradually get more complex throughout.

Part 1 – Project Setup

Setup

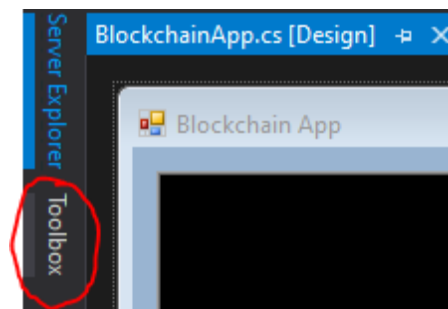
Download the BlockchainAssignment Project from Blackboard. Open Visual Studio click open project/solution and select *BlockchainAssignment.sln*. In the Solution explorer you will see the BlockchainAssignment.csproj. Expand this to find the components of the project, including the code, references, and properties. For us, the important parts are the *BlockchainApp.cs*, *HashCode-*, and *Wallet-* folders.

Double click the *BlockchainApp.cs* to open a Windows Form, which shows the basic UI of the App. You are free to modify it during this assignment as you see fit but ensure that the results are clear (as you are required to provide screenshots in your report). Right click *BlockchainApp.cs* and click *view code* to see the underlying code behind the UI. Currently it is mostly empty.

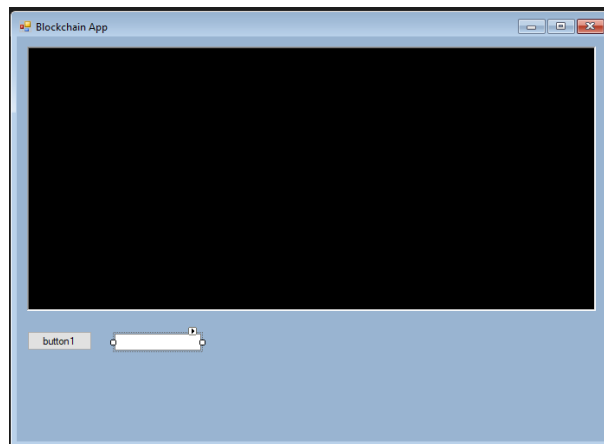
The *HashCode* and *Wallet* folders are host to ready-made code that will be applied later in this assignment. *Wallet* is responsible for the asymmetric encryption (public and private key pairs) and digital signatures in transaction. *HashCode* is responsible for combining hashes and converting hashes to and from byte arrays to strings both ways. This code is provided to help reduce the workload of the project, such that time can be evenly distributed across each of the components.

Customising the UI

To familiarise yourself with Windows Forms and C# briefly add some functionality to the UI. First add a button, on the left side of the screen a horizontal tab titled *Toolbox* should be visible.



Click *Toolbox* and a window will expand, click *All Windows Forms* or *Common Controls*, and click and drag *Button* onto the UI. Now search for a *TextBox* in the toolbox and drag it onto the UI. Your UI should now appear as the following:

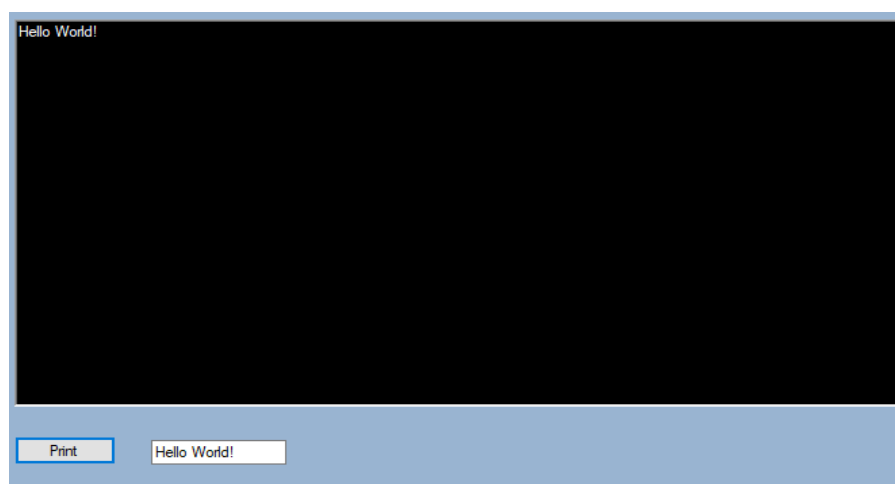


Click *button1* once to open its properties in the bottom left. Change its *text* property to say '*Print*' instead of '*button1*'. Now double click the button and you will be taken to the underlying code, a new function for clicking the button is created after double clicking it. Next you can write some code that takes the text from the *TextBox* we added and prints it to the larger textbox above, when we click the *Button*. To get the names of the Textboxes you can click them in the UI form, in this case the large black text box is *richTextBox1* and the smaller text box is *textBox1*.

Code:

```
1 reference
private void button1_Click(object sender, EventArgs e)
{
    richTextBox1.Text = textBox1.Text;
}
```

Output:



Note: This textbox and button can be removed or re-purposed after this section as you wish.